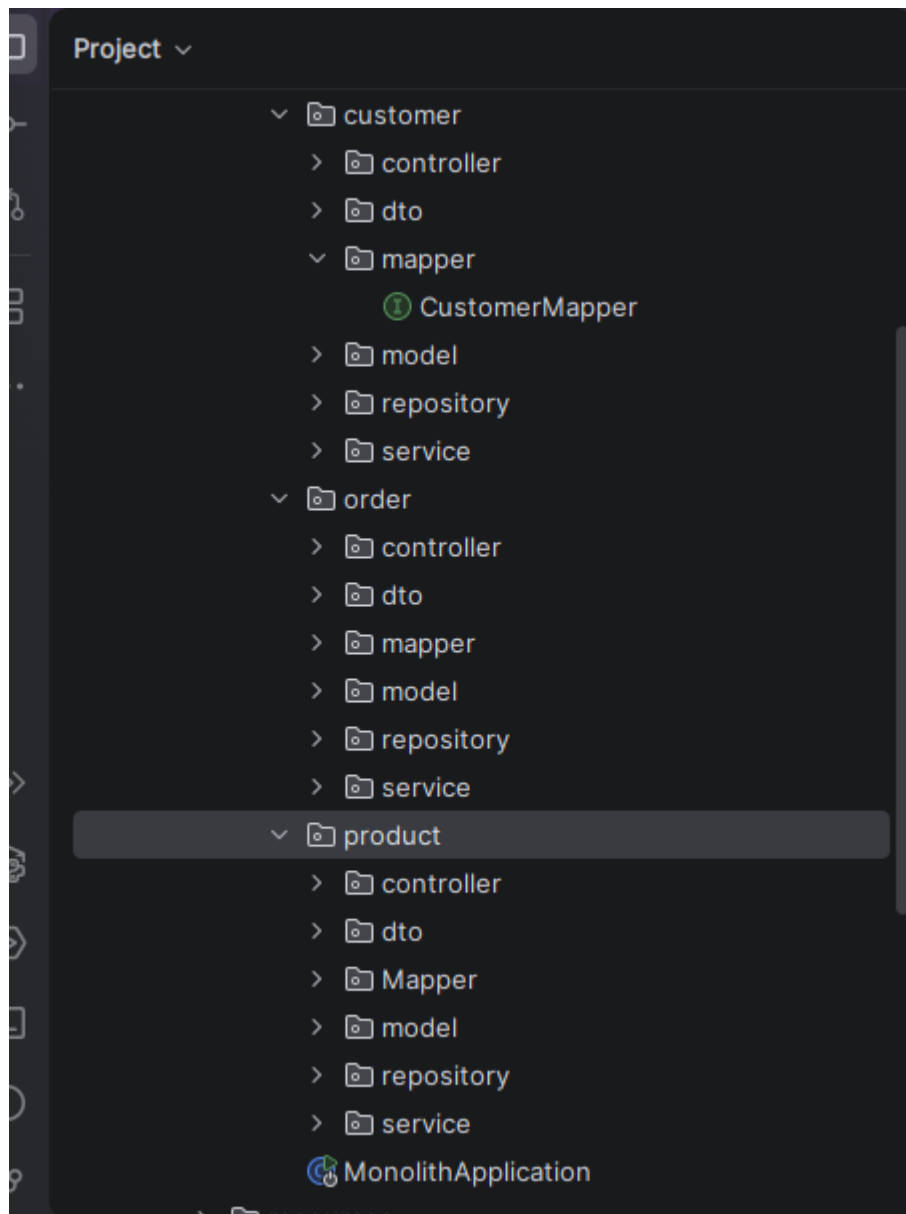


TP2 : Monolithe Modulaire

Objectifs :

- Restructurer le monolithe en modules métier distincts
- Définir des boundaries claires entre domaines
- Implémenter des interfaces de communication entre modules
- Appliquer les design patterns (DTO, Mapper, Service Layer)
- Préparer le terrain pour une future migration microservices

Étape 1 : Architecture Modulaire (Structure du Projet)



Étape 2 : Couche de Transfert de Données (DTO & MapStruct)

```
OrderDTO.java x
1 package com.ecommerce.monolith.order.dto;
2
3 > import ...
4
5
6
7 @Data 25 usages mohammedkhamrich
8 public class OrderDTO {
9     private Long id;
10    private Long customerId;
11    private Long productId;
12    private Integer quantity;
13    private BigDecimal totalPrice;
14    private LocalDateTime orderDate;
15 }
```

Productmapper.java

```
1 package com.ecommerce.monolith.product.mapper;
2
3 import com.ecommerce.monolith.product.dto.CreateProductRequest;
4 import com.ecommerce.monolith.product.dto.ProductDTO;
5 import com.ecommerce.monolith.product.model.Product;
6 import org.mapstruct.*;
7 import java.util.List;
8
9 @Mapper(componentModel = "spring") 3 usages 1 implementation mohammedkhamrich
10 public interface ProductMapper {
11     ProductDTO toDTO(Product product); 4 usages 1 implementation mohammedkhamrich
12     List<ProductDTO> toDTOList(List<Product> products); 1 usage 1 implementation mohammedkha
13     Product toEntity(CreateProductRequest request); 1 usage 1 implementation mohammedkhamrich
14
15     @Mapping(target = "id", ignore = true) 1 usage 1 implementation mohammedkhamrich
16     void updateEntity(CreateProductRequest request, @MappingTarget Product product);
17 }
```

Étape 3 : Logique Inter-Modules (Order Service)

```
OrderDTO.java  ProductMapper.java  OrderServiceImpl.java x
package com.ecommerce.monolith.order.service;

import ...

@Service  mohammedkhamrich
@RequiredArgsConstructor
@Transactional
public class OrderServiceImpl implements OrderService {

    private final OrderRepository orderRepository;
    private final OrderMapper orderMapper;
    private final ProductService productService;
    private final CustomerService customerService;

    @Override 1 usage  mohammedkhamrich
    public OrderDTO createOrder(CreateOrderRequest request) {
        if (!customerService.existsById(request.getCustomerId()))
            throw new EntityNotFoundException("Customer not found: " + request.getCustomerId());

        ProductDTO product = productService.getProductById(request.getProductId());

        BigDecimal total = product.getPrice()
            .multiply(BigDecimal.valueOf(request.getQuantity()));

        Order order = Order.builder()
```

Étape 6 : Gestion des Versions (GitHub)

main 1 Branch 0 Tags			Go to file	Add file	Code
mohammedkhamrich Finalisation du TP2 : Architecture modulaire, DTOs et Mappers fon... 1234402 · 36 minutes ago 2 Commits					
TP1-Monolithe-Simple	initialisation de depot	3 weeks ago			
TP2-Monolithe-Modulaire- Copie	Finalisation du TP2 : Architecture modulaire, DTOs et Mapper...	36 minutes ago			